

Herald

每条警报都能精准送达——无需命令语法。

概述

Herald 接收系统事件，并可靠地将其分发至多个通知渠道，确保每条警报都能准确送达目的地。订阅者使用自然语言交互，Herald 通过三层机制推断意图（命令快速路径 → LLM 意图推理 → 确认回退）。

简要说明

事件接收与多渠道通知分发系统。Herald 可靠地将系统事件路由至各类消息渠道的正确目的地，并允许订阅者使用自然语言交流——通过命令快速路径、LLM 推理及确认回退机制解析意图。

详细说明

Herald 是通知系统的核心支柱，确保系统事件能真正送达可执行操作的人员手中——这一看似平凡却至关重要的环节，往往是自建告警系统悄然失效的根源。它接收事件并可靠地分发至多个通知渠道，杜绝常见的失效模式：警报丢失、误投至无效渠道，或淹没在噪声中直至为时已晚。然而，可靠送达仅仅是问题的一半；另一半则是人类如何响应。Herald 拒绝传统的妥协方案，即要求用户死记硬背僵化的命令语法与告警机器人交互。订阅者只需使用自然语言，Herald 便通过精心设计的三层机制解析其意图：首先是快速路径，瞬间识别明确的命令；其次是基于 LLM（通过 Claude Code 实现）的意图推理，处理自由形式的消息；最后则是“确认”回退机制，在意图真正模糊时回复、标记并提问。这一“识别 → 推理 → 确认”的阶梯式设计，正是整个系统理念的缩影——常见场景保持即时且确定性，灵活场景由模型处理，而不确定场景绝不会靠盲目猜测触发错误操作。

Herald 还建模了参与者与归属关系：通过操作员用户名环境变量（`HERALD_<CHANNEL>_OPERATOR_USERNAME`）及参与者/归属契约，驱动 `created_by/assigned_to` 字段和通知中的 @ 标记，清晰记录谁做了什么、谁被通知。在治理层面，Herald 继承 Helix Constitution 作为同地部署的子模块，并遵循其规则；同时，它也是 Docs Chain 的早期生产级用户——其完整的 66 份 Markdown→HTML/PDF/DOCX 语料库，均通过 Docs

Chain 的 exec：转换进行处理并验证无误。Herald 主要基于 Shell/Go 工具构建，采用分层规范（V1→V2→V3→V4 迭代升级），并为各渠道提供操作员设置指南，适用于消息平台及 LLM/代理分发器。

为何开发

警报常常悄然失效——被发送至错误渠道、遗漏，或要求用户记住僵化的命令语法。Herald 的诞生，正是为了确保可靠的分发机制，并让用户能用自然语言响应，使通知既可靠又易于操作。

内容

为何堪称变革性产品

它将两项通常作为独立产品购买的功能——可靠的多渠道事件路由与自然语言交互界面——融为一体，操作人员只需开口交流，系统便能理解其意图。而其明确的容错机制正是让系统在生产环境中值得信赖的关键：一个宁可主动询问而非误报的警报系统，才是真正能够接入实际业务的系统。

创新之处

- 三层意图识别机制：快速命令通道 → LLM 推理层 → 确认询问流程。
- 自然语言订阅交互（无需学习命令语法）。
- 参与者归属协议，驱动 created_by/assigned_to 字段及 @ 标记功能。
- 真实 Docs Chain 消费者（66 份文档语料库，多格式支持，验证通过）。

挑战与解决方案

- 自然语言意图模糊性：通过三层识别/推理/确认阶梯解决，避免盲目猜测。
- 可靠的消息分发：采用摄取→多渠道分发设计，确保警报送达正确目的地。
- 跨渠道归属准确性：通过操作员用户名环境变量与参与者归属协议解决。
- 文档版本偏移：通过 Docs Chain 将文档语料库纳入验证流程，确保转换准确无误。

技术栈（选型依据与实现方式）

- **Go** —— 核心事件/分发逻辑（按组织语言模式定制）。
- **Shell** —— 操作员工具及配置脚本。
- **Claude 代码（LLM）** —— 自由消息的意图推理层。
- 消息渠道适配器 —— 多渠道通知分发。

- **Docs Chain** —— 文档构建/验证流水线 (Markdown → HTML/PDF/DOCX) 。
- **Helix Constitution** 子模块 —— 继承治理与规则体系。